μ, the current running average. We can easily store this in a 10 × 2 numpy array (assuming we have 10 arms). We'll call this array the `record`.

```
>>> record = np.zeros((n,2))
array([[0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.]])
```

The first column of this array will store the number of times each arm has been pulled, and the second column will store the running average reward. Let's write a function for updating the record, given a new action and reward.

Listing 2.4. Updating the reward record

```
def update_record(record,action,r):
    new_r = (record[action,0] * record[action,1] + r) /
(record[action,0] +
     1)
    record[action,0] += 1
    record[action,1] = new_r
    return record
```

This function takes the `record` array, an action (which is the index value of the arm), and a new reward observation. To update the average reward, it simply implements the mathematical function we described previously, and then increments the counter recording how many times that arm has been pulled.

Next we need a function that will select which arm to pull. We want it to choose the arm associated with the highest average reward, so all we need to do is find the row in the `record` array with biggest value in column 1. We can easily do this using numpy's built-in `argmax` function, which takes in an array, finds the largest value in the array, and returns its index position.